

SEN 2008 — Software Security Term Project

Secure Mobile Authentication with OAuth 2.0 Authorization Code Flow and PKCE (iOS + Java Backend)

Group Members

Full Name	Student ID
Batuhan Arda	2368567
Abdullabek Atajanov	2364274
Kerem Gürcan	2366446

1. Topic Identification — The Problem and the Gap

Modern applications increasingly delegate authentication and authorization to the OAuth 2.0 framework. A particularly important and security-sensitive case is the **public client**: a mobile or single-page application that runs entirely on a device the user controls. Unlike a server-side "confidential" client, a public client cannot safely store a long-lived secret, because anything shipped inside the app binary can be extracted by an attacker.

The OAuth 2.0 Authorization Code grant was originally designed assuming the client could authenticate itself with a client secret. On mobile platforms this assumption breaks down, which exposes a concrete and well-documented attack: the **authorization code interception attack**. On a mobile device, the authorization server redirects back to the app using a custom URL scheme (e.g. `com.example.oauthdemo://callback?code=...`). A malicious application installed on the same device can register the same scheme and intercept the authorization code in transit. With the intercepted code alone, on a server that does not implement additional protections, the attacker can complete the token exchange and impersonate the user.

The gap our project addresses: how to make the Authorization Code grant safe for public clients that cannot keep a secret. The answer adopted by the security community is **Proof Key for Code Exchange (PKCE, RFC 7636)**, which cryptographically binds an authorization code to the specific client instance that requested it. Our project is a direct implementation of the Authorization Code + PKCE flow, plus a live demonstration that proves PKCE neutralizes the interception attack.

2. Literature Review

We selected and critically analyzed four authoritative technical sources. All are IETF standards-track or informational RFCs, which are the canonical references for this domain.

2.1 RFC 6749 — The OAuth 2.0 Authorization Framework

RFC 6749 defines the core OAuth 2.0 roles (resource owner, client, authorization server, resource server) and grant types, including the Authorization Code grant our project builds on. Its strength is a clean separation between obtaining an authorization code and exchanging it for an access token. Its principal limitation, from a modern security standpoint, is that several of its protections (notably client authentication at the token endpoint) implicitly assume a confidential client. For public clients this assumption does not hold, leaving the bare RFC 6749 Authorization Code grant vulnerable to code interception. This limitation is precisely what motivated the later PKCE specification.

2.2 RFC 7636 — Proof Key for Code Exchange (PKCE)

RFC 7636 is the foundational source for our project. It introduces a per-request secret, the `code_verifier`, and a derived public value, the `code_challenge`. The client commits to the verifier at the `/authorize` step (by sending only the challenge) and proves possession of it at the `/token` step (by sending the verifier itself). The server recomputes the challenge from the presented verifier and rejects the exchange if they do not match. The strength of this approach is that it requires no pre-shared secret and no additional infrastructure, making it ideal for public clients. A practical limitation is that PKCE protects only the code-to-token exchange; it must be combined with other controls (TLS, the `state` parameter, single-use codes) to provide complete protection.

2.3 RFC 7519 — JSON Web Token (JWT)

RFC 7519 defines a compact, self-contained, digitally signed token format. We use JWT as the access-token format issued by our token endpoint. Its strength is statelessness: the resource endpoint can validate a token by checking the signature and expiry without a database lookup. Its main risks — which our implementation respects — are weak signing keys and excessive token lifetimes; we use an HMAC-SHA256 signature and a short expiry.

2.4 RFC 9700 — Best Current Practice for OAuth 2.0 Security (2025)

RFC 9700 consolidates current OAuth security guidance and reflects how the field has evolved since 2012. Its most relevant directive for us is that PKCE is now recommended for essentially all Authorization Code exchanges, not just public clients. This source establishes that our design choice is aligned with the current state of the art rather than a historical curiosity.

Critical synthesis. Reading these sources together tells a coherent story: RFC 6749 establishes a flow that is safe only under an assumption (confidential clients) that

mobile apps violate; RFC 7636 repairs that flow for public clients; RFC 7519 supplies the token format; and RFC 9700 confirms that PKCE has become baseline best practice. Our project implements exactly this corrected, modern flow and demonstrates the concrete attack it prevents.

3. Foundational Analysis — Deep Dive into RFC 7636 (PKCE)

3.1 Core Problem

The specific vulnerability PKCE addresses is the **authorization code interception attack** against public clients. Because a mobile client cannot hold a secret and because the redirect back to the app can be intercepted by a malicious app registered for the same URL scheme, an attacker who captures the authorization code can redeem it for tokens. The core problem is therefore: *how can a client prove, at the token endpoint, that it is the same entity that initiated the authorization request — without relying on a stored secret?*

3.2 Technical Approach

PKCE solves this with a one-time, dynamically generated secret:

1. **Generate a code_verifier** — a high-entropy random string created freshly for each authorization request and kept only in the client's memory.
2. **Derive a code_challenge** — using the S256 method, `code_challenge = BASE64URL(SHA256(code_verifier))`. Only the challenge (not the verifier) is sent to the authorization server in the `/authorize` request, and the server stores it alongside the issued authorization code.
3. **Prove possession at /token** — when exchanging the code, the client sends the original `code_verifier`. The server recomputes `BASE64URL(SHA256(code_verifier))` and compares it to the stored challenge. If they match, the caller demonstrably holds the secret that initiated the request; if not, the exchange is rejected.

Because the verifier never leaves the legitimate client and the challenge is a one-way hash, an attacker who intercepts only the authorization code cannot produce a matching verifier and therefore cannot complete the exchange.

3.3 Key Findings and Significance

The significance of PKCE is that it makes the Authorization Code grant safe for clients that cannot keep a secret, using only a hash function and a random number generator — no extra round trips, no shared secrets, no new infrastructure. Originally introduced for mobile/native apps, it has since been generalized (per RFC 9700) to a recommended default for all clients, including web applications. It is one of the clearest examples in applied security of a small, well-targeted cryptographic binding eliminating an entire class of attacks.

4. Technical Implementation

4.1 Architecture Overview

The system has two components communicating over HTTP(S):

- **iOS client (Swift / SwiftUI):** a public client that initiates the flow, generates the PKCE secret, performs the authorization step in a secure system browser, exchanges the code for a token, stores the token, and calls a protected resource.
- **Java backend (Spring Boot):** a minimal authorization + resource server exposing three endpoints — `/authorize`, `/token`, and `/me` — with all state held in memory for the purposes of this demonstration.

The end-to-end flow is: the app generates a `code_verifier/code_challenge`, opens `/authorize` (which auto-approves a single hardcoded demo user and 302-redirects back with an authorization code), the app exchanges the code at `/token` together with the `code_verifier`, the backend verifies PKCE and returns a signed JWT, and the app then calls the protected `/me` endpoint with the JWT as a Bearer token.

4.2 Backend (Spring Boot 3.x, Java 17)

Key classes and responsibilities:

- **AuthController** — exposes `/authorize`, `/token`, and `/me`.
- GET `/authorize` validates `client_id`, `redirect_uri`, and that `code_challenge_method=S256`, generates a single-use authorization code bound to the supplied `code_challenge`, and returns a 302 redirect to the registered `redirect_uri` with `code` and `state` appended. Validation failures return clean JSON errors rather than redirecting.
- POST `/token` looks up the code, confirms it is unexpired and unused, verifies PKCE by recomputing `BASE64URL(SHA256(code_verifier))` and comparing it to the stored challenge, then (on success) invalidates the code and issues a signed JWT.
- GET `/me` requires an `Authorization: Bearer <jwt>` header, validates the signature and expiry, and returns the demo user's identity.
- **CodeStore / AuthorizationCode** — in-memory store of issued codes with secure-random code generation, a short time-to-live (60 seconds), and a single-use flag.
- **PkceVerifier** — computes the S256 challenge and performs a **constant-time comparison** against the stored challenge to avoid timing side channels.
- **JwtService** — issues and validates HS256 JWTs with a short (5-minute) expiry.
- **DemoConfig** — centralizes all hardcoded demo values (client id, redirect URI, demo user, signing secret, TTLs).

4.3 iOS Client (SwiftUI, iOS 16+)

The OAuth/PKCE logic is deliberately isolated from the user interface:

- **PKCEHelper** — generates a high-entropy `code_verifier` and derives the S256 `code_challenge` using Apple's CryptoKit.

- **KeychainHelper** — stores, reads, and deletes the JWT in the iOS Keychain (secure credential storage rather than plain UserDefaults).
- **AuthManager** (an ObservableObject) — owns the entire flow: it builds the /authorize URL with the challenge and a random state, launches an **ASWebAuthenticationSession** (the platform's secure, sandboxed browser for auth), captures the redirect, **verifies the returned state** matches the one it sent, extracts the code, POSTs it to /token with the code_verifier, stores the resulting JWT, and exposes published state to the UI.
- **ContentView** — a thin SwiftUI view that only observes AuthManager (Sign in / Call / me / Logout). All UI updates are marshalled to the main thread.

The project is generated reproducibly from a declarative `project.yml` via **XcodeGen**, which also injects the required `Info.plist` settings (custom URL scheme registration and a narrowly scoped App Transport Security exception for localhost).

4.4 Security Mechanisms (summary)

- **PKCE (S256)**: binds the authorization code to the client instance; defeats code interception.
- **state parameter**: mitigates cross-site request forgery / response-mixing on the redirect.
- **Single-use, short-TTL authorization codes**: limit the window and reuse of an intercepted code.
- **Constant-time challenge comparison**: avoids timing side channels in PKCE verification.
- **Signed JWT with short expiry**: stateless, tamper-evident access tokens with limited lifetime.
- **Keychain storage**: protects the token at rest on the device.
- **ASWebAuthenticationSession (ephemeral)**: isolates the authorization step in a trusted, sandboxed browser context.
- **Transport**: HTTPS in production; for the local demo a narrowly scoped ATS exception permits `http://localhost` only (see Limitations).

5. Innovation and Contribution

Our original contribution goes beyond a straightforward implementation of the RFC: we built a **live, scripted interception-attack demonstration** that empirically proves what PKCE accomplishes, and we made a deliberate, defensible security-design decision that the demo validates.

5.1 A demonstrable attack harness (vulnerable vs. protected, side by side)

To make the protection tangible, the backend exposes one additional, **clearly labelled, demo-only endpoint** (`/token-legacy-insecure`) that simulates a pre-PKCE authorization server: it exchanges a code for a token **without** checking any `code_verifier`. The real `/token` endpoint is left untouched and continues to enforce

PKCE. A single Python script (standard library only) then runs three scenarios against the live backend, using one intercepted authorization code throughout:

1. **Vulnerable server (no PKCE):** the attacker presents the stolen code to the legacy endpoint and **receives a valid token** — *attack succeeds*.
2. **PKCE-protected server (attacker):** the attacker presents the same stolen code to the real `/token` with no/wrong verifier and is **rejected with invalid_grant** — *attack blocked*.
3. **PKCE-protected server (legitimate client):** the real client presents the same code with the correct verifier and **succeeds** — *legitimate flow still works*.

This produces a clear, audience-readable narrative that demonstrates both the vulnerability and the fix in one run.

5.2 A deliberate design decision: do not consume the code on a failed PKCE check

We made a conscious choice that a **failed** PKCE verification does **not** invalidate the authorization code. The rationale is denial-of-service resistance: if a failed attempt burned the code, an attacker who intercepted it could intentionally submit a wrong verifier to invalidate it before the legitimate client redeems it, denying service to the real user. By not consuming the code on failure (and relying on the short TTL plus single-use-on-success), an intercepted code without the verifier simply expires harmlessly while the legitimate client can still use it. Scenario 3 above empirically confirms this: the same code that the attacker failed to redeem in Scenario 2 is still successfully redeemed by the legitimate client.

6. Limitations and Demo Simplifications

In the interest of academic honesty, the following are deliberate simplifications made to keep the project focused on the PKCE security property rather than on production hardening:

- **Single hardcoded user and client**, with **in-memory state** and no database.
 - **Auto-approval at `/authorize`** — no login/consent UI is rendered; the security focus is the PKCE/code exchange, not the authentication UI.
 - **Hardcoded JWT signing secret** kept in one clearly marked constant; production would use a managed, rotated key.
 - **`/token-legacy-insecure` is intentionally vulnerable** and exists solely to demonstrate the pre-PKCE attack; it must never appear in a real system. For demonstration convenience it does not consume the code, so a single intercepted code can flow through all three scenarios; single-use is an independent control and the property being demonstrated is specifically the missing PKCE check.
 - **Plaintext HTTP to localhost** via a narrow ATS exception for the Simulator; production uses HTTPS exclusively.
-

7. Individual Contributions

The work was divided evenly across the three members, with all design decisions made jointly.

Member	Role and Responsibilities
Batuhan Arda	iOS / SwiftUI client implementation (AuthManager, PKCE generation with CryptoKit, ASWebAuthenticationSession flow, Keychain storage, UI) and overall system architecture and client-server integration.
Abdullabek Atajanov	Java / Spring Boot backend (/authorize, /token, /me endpoints, in-memory code store, JWT issuance and validation, PKCE verification logic) and end-to-end testing of the flow.
Kerem Gürcan	Security mechanisms and the interception-attack demonstration (legacy-vs-PKCE comparison script), together with the literature review and the compilation of the technical report.

All members participated in the design decisions, are fully conversant with both the theoretical background and the code implementation, and contributed to testing and the live demonstration.

8. References

1. D. Hardt (ed.), *The OAuth 2.0 Authorization Framework*, RFC 6749, IETF, 2012.
2. N. Sakimura, J. Bradley, N. Agarwal, *Proof Key for Code Exchange by OAuth Public Clients*, RFC 7636, IETF, 2015.
3. M. Jones, J. Bradley, N. Sakimura, *JSON Web Token (JWT)*, RFC 7519, IETF, 2015.
4. T. Lodderstedt, J. Bradley, A. Labunets, D. Fett, *Best Current Practice for OAuth 2.0 Security*, RFC 9700, IETF, 2025.